

Perfect — I adjusted DAY 13 exactly to YOUR folder structure  (no mismatch)

DAY 13 — EVALUATION + LOGGING (YOUR STRUCTURE VERSION)

STEP 1 — CREATE FOLDER

Inside your root:

mutimodel_llm/

 evaluation/

 logs/

STEP 2 — CREATE FILES

 evaluation/evaluator.py

```
from models.llm import generate
```

```
class Evaluator:
```

```
    def init(self):
```

```
        pass
```

```
    def evaluate(self, query, context, answer):
```

```
        prompt = f"""
```

You are an AI evaluator.

Score the answer based on:

1. Relevance (0-10)

2. Faithfulness (0-10)

3. Correctness (0-10)

Context:

{context}

Question:

{query}

Answer:

{answer}

Return ONLY this format:

Relevance: X

Faithfulness: X

Correctness: X

"""

```
result = generate(prompt)
```

```
return result
```

 evaluation/logger.py

```
import json
```

```
import os
```

```
class Logger:
```

```
def init(self, path="logs/history.json"):
    self.path = path
    os.makedirs(os.path.dirname(self.path), exist_ok=True)
```

```
def log(self, data):
```

```
    if not os.path.exists(self.path):
```

```
        logs = []
```

```
    else:
```

```
        with open(self.path, "r") as f:
```

```
            logs = json.load(f)
```

```
    logs.append(data)
```

```
    with open(self.path, "w") as f:
```

```
        json.dump(logs, f, indent=2)
```

```
``
```

✅ STEP 3 — UPDATE YOUR CURRENT AGENT

 app/agent.py

✅ ADD IMPORTS

```
from evaluation.evaluator import Evaluator
```

```
from evaluation.logger import Logger
```

```
``
```

✅ UPDATE __init__

class Agent:

```
def init(self, rag_system, excel_tool=None, pdf_table_tool=None):
```

```
    self.rag = rag_system
```

```
    self.excel = excel_tool
```

```
    self.pdf_table = pdf_table_tool
```

```
    self.planner = Planner()
```

```
    self.memory = ChatMemory()
```

```
    self.evaluator = Evaluator() # ✅ NEW
```

```
    self.logger = Logger()      # ✅ NEW
```

✅ MODIFY FINAL PART OF run()

(leave everything before same — only replace ending)

```
# Step: final answer
```

```
final_answer = generate(final_prompt)
```

```
# ✅ Evaluation
```

```
evaluation = self.evaluator.evaluate(
```

```
    query=query,
```

```
    context=str(results),
```

```
    answer=final_answer
```

```
)
```

```
print("\nEVALUATION:\n", evaluation)
```

```
#  Logging
```

```
self.logger.log({  
    "query": query,  
    "steps": results,  
    "answer": final_answer,  
    "evaluation": evaluation  
})
```

```
#  Memory
```

```
self.memory.add(query, final_answer)
```

```
return final_answer
```

FINAL FLOW (YOUR SYSTEM)

User Query



Planner



Tools (RAG / Excel / PDF_TABLE)



Results



LLM Answer



 Evaluator (NEW)



✓ Logger (NEW)



Memory update

✓ TEST

Run:

`python -m app.main`

Try:

>> what is ai

>> average sales

>> summarize report

✓ OUTPUT

Answer: ...

EVALUATION:

Relevance: 9

Faithfulness: 8

Correctness: 9

✓ LOG FILE CREATED

logs/history.json

Stores:

- query
- steps (tool outputs)
- answer

- evaluation score
-

✅ WHAT YOU JUST ADDED

Evaluation Layer

Logging Layer

✅ SYSTEM LEVEL NOW

You now have:

- RAG ✅
 - Excel Agent ✅
 - PDF Table Agent ✅
 - Planner ✅
 - Multi-step reasoning ✅
 - Memory ✅
 - ✅ Evaluation
 - ✅ Logging
-

✅ NEXT

DAY 14 → FastAPI + UI (make it a real product)

If you want next, I'll give: 👉 DAY 14 in same exact format (connect UI + API)